

# MÁQUINA DE ESTADOS COM DOIS AGENTES

## Guarda e Sistema de Alarme

### 1. Visão geral do cenário (mínimo 1 parágrafo contextualizando).

O projeto simula um ambiente e a segurança feita por dois agentes de comportamentos diferentes e funcionam por meio de máquinas de estados. O Agente A é um guarda que pode patrulhar o local, entrar em alerta e investigar alguma coisa que chama a atenção dele. Já o Agente B é um sistema de alarme, que monitora os sensores, aciona o alarme quando necessário e depois encerra o ciclo do alarme. Os ciclos acontecem chamados pela classe GerenciadorMaquina. Durante a execução, cada agente possui um estado atual, que define o que ele deve fazer naquele momento e quando deve passar para outro estado. A lógica de cada comportamento fica separada em classes específicas, deixando o código mais organizado e evitando o uso de vários if e else concentrados em um único lugar.

### 2. Visão geral de cada agente (mínimo 1 parágrafo por agente).

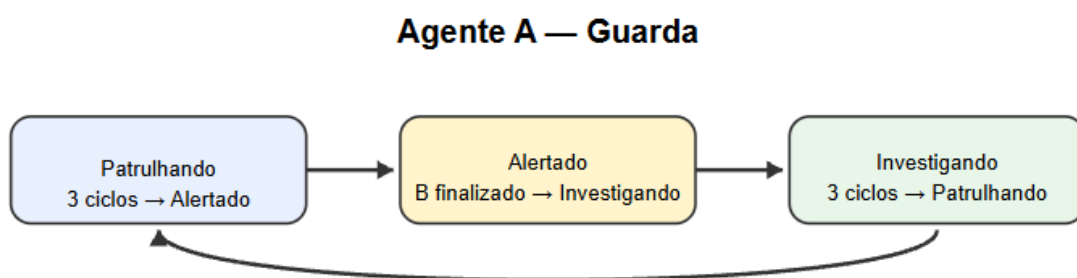
#### 2.1 Agente A – Guarda

O Agente A atua como o guarda do local. Ele inicia a simulação em sua patrulha, mantendo esse ritmo por três ciclos. Em seguida, entra em estado de alerta, reagindo como se tivesse notado algo estranho. Durante o alerta, ele espera um sinal do sistema de alarme. Assim que é notificado de que o alarme concluiu o ciclo, o guarda passa para a etapa de investigação, onde dedica mais três ciclos para checar a ocorrência antes de retornar à patrulha normal. Para gerenciar o ciclo, o agente monitora seu estado atual, usa um contador de ciclos e conta com a variável booleana `alarmeFinalizado`, que recebe do Gerenciador o aviso de encerramento do alarme.

#### 2.2 Agente B – Sistema de Alarme

O Agente B é o sistema de alarme da simulação. Ele começa no estado `AlarmeInativo`, monitorando os sensores esperando qualquer movimento. Após três ciclos de monitoramento, o alarme dispara e o agente entra no estado `AlarmeDisparado`. O alarme toca continuamente por três ciclos até que o sistema transite para o estado `AlarmeFinalizado`.

### 3. Diagrama de estados por agente (imagem inserida no documento).



## Agente B — Alarme



4. Tabela de regras por agente (estado, ações de entrada/execução/saída, transições).

### Agente A

|              |                                                  |                                                                               |                                    |                                                                    |
|--------------|--------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------|
| Patrulhando  | Zera contador.                                   | Incrementa contador e registra patrulha.                                      | Informa encerramento da patrulha.  | Após contador $\geq 3$ , vai para Alertado.                        |
| Alertado     | Zera contador.                                   | Verifica alarmeFinalizado; enquanto for falso, incrementa contador e aguarda. | Informa fim do alerta.             | Quando alarmeFinalizado $\Rightarrow$ true, vai para Investigando. |
| Investigando | Zera contador e redefine alarmeFinalizado=false. | Incrementa contador e procura a origem do alerta.                             | Informa conclusão da investigação. | Após contador $\geq 3$ , volta para Patrulhando.                   |

### Agente B

|                  |                |                                              |                                                 |                                                     |
|------------------|----------------|----------------------------------------------|-------------------------------------------------|-----------------------------------------------------|
| AlarmeInativo    | Zera contador. | Incrementa contador e verifica sensores.     | Registra detecção de anomalia.                  | Após contador $\geq 3$ , vai para AlarmeDisparado.  |
| AlarmeDisparado  | Zera contador. | Incrementa contador e mantém a sirene ativa. | Registra fim do ciclo da sirene.                | Após contador $\geq 3$ , vai para AlarmeFinalizado. |
| AlarmeFinalizado | Zera contador. | Informa que aguarda novo ciclo.              | Prevista mensagem de reinício do monitoramento. | Não há transição automática implementada.           |

5. Descrição das variáveis/limiares por agente (nome, tipo, intervalo, valor inicial, limiares para troca de estados, etc).

|       |                  |         |               |                                                           |                                                                          |
|-------|------------------|---------|---------------|-----------------------------------------------------------|--------------------------------------------------------------------------|
| A     | contadorEstado   | int     | 0             | Inteiro não negativo; conta execuções no estado atual.    | 3 em Patrulhando e 3 em Investigando. Em Alertado não determina a troca. |
| A     | alarmeFinalizado | boolean | false         | false/true; sinal de comunicação recebido do Gerenciador. | true provoca Alertado -> Investigando.                                   |
| A     | estadoAtual      | Estado  | Patrulhando   | Uma implementação da interface Estado.                    | Alterado por mudarEstado(...).                                           |
| B     | contadorEstado   | int     | 0             | Inteiro não negativo; conta execuções no estado atual.    | 3 em Inativo e 3 em Disparado.                                           |
| B     | estadoAtual      | Estado  | AlarmeInativo | Uma implementação da interface Estado.                    | Alterado por mudarEstado(...).                                           |
| Geral | ciclos           | int     | 12 na Main    | Quantidade de iterações do loop principal.                | 0 for executa de 1 até ciclos.                                           |

6. Estrutura do código (quais são as classes/arquivos do projeto e qual o papel de cada uma delas dentro do projeto).

|                                     |                                                                                                                                                       |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/Main.java                       | Ponto de entrada. Cria AgenteA e AgenteB, cria o GerenciadorMaquina e executa 12 ciclos.                                                              |
| src/agentes/Estado.java             | Interface que define o contrato comum dos estados: enter(Agente), execute(Agente) e leave(Agente).                                                    |
| src/agentes/Agente.java             | Classe abstrata base. Armazena nome e estadoAtual; centraliza atualizar() e mudarEstado(), incluindo leave do estado anterior e enter do novo estado. |
| src/agentes/AgenteA.java            | Representa o Guarda. Mantém contadorEstado e alarmeFinalizado e começa em EstadoPatrulhando.                                                          |
| src/agentes/AgenteB.java            | Representa o Alarme. Mantém contadorEstado e começa em EstadoAlarmeInativo.                                                                           |
| EstadoPatrulhando.java              | Implementa a patrulha do Agente A e a transição para Alertado após 3 ciclos.                                                                          |
| EstadoAlertado.java                 | Mantém o Guarda aguardando a flag alarmeFinalizado; ao recebê-la, muda para Investigando.                                                             |
| EstadoInvestigando.java             | Executa a investigação por 3 ciclos, limpa a flag do alarme e retorna à patrulha.                                                                     |
| EstadoAlarmeInativo.java            | Simula monitoramento dos sensores por 3 ciclos e dispara o alarme.                                                                                    |
| EstadoAlarmeDisparado.java          | Mantém a sirene ativa por 3 ciclos e depois finaliza o alarme.                                                                                        |
| EstadoAlarmeFinalizado.java         | Estado terminal do ciclo do alarme; permanece aguardando, sem reinicialização automática.                                                             |
| src/maquina/GerenciadorMaquina.java | Executa o loop. Atualiza B e A nessa ordem e, ao detectar B em EstadoAlarmeFinalizado, define alarmeFinalizado=true em A.                             |

## 7. Resultados esperados (exemplos de logs de transição).

[Agente A] ENTRADA: iniciando patrulha.

[Agente B] ENTRADA: alarme inativo e monitorando sensores.

===== CICLO 1 =====

[Agente B] EXECUÇÃO: verificando sensores... (1/3)

[Agente A] EXECUÇÃO: patrulhando... (1/3)

===== CICLO 2 =====

[Agente B] EXECUÇÃO: verificando sensores... (2/3)

[Agente A] EXECUÇÃO: patrulhando... (2/3)

===== CICLO 3 =====

[Agente B] EXECUÇÃO: verificando sensores... (3/3)

[Agente B] SAÍDA: anomalia detectada; disparando alarme.

[Agente B] ENTRADA: sirene ativada.

[Agente A] EXECUÇÃO: patrulhando... (3/3)

[Agente A] SAÍDA: comportamento de patrulha encerrado.

[Agente A] ENTRADA: guarda em estado de alerta.

===== CICLO 4 =====

[Agente B] EXECUÇÃO: sirene tocando... (1/3)

[Agente A] EXECUÇÃO: aguardando finalização do alarme... (1/2)

===== CICLO 5 =====

[Agente B] EXECUÇÃO: sirene tocando... (2/3)

[Agente A] EXECUÇÃO: aguardando finalização do alarme... (2/2)

===== CICLO 6 =====

[Agente B] EXECUÇÃO: sirene tocando... (3/3)

[Agente B] SAÍDA: ciclo da sirene finalizado.

[Agente B] ENTRADA: alarme finalizado.

[Agente A] EXECUÇÃO: aguardando finalização do alarme... (3/2)

===== CICLO 7 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] SAÍDA: alerta encerrado; iniciando investigação.

[Agente A] ENTRADA: investigando a área.

===== CICLO 8 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] EXECUÇÃO: procurando a origem do alerta... (1/3)

===== CICLO 9 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] EXECUÇÃO: procurando a origem do alerta... (2/3)

===== CICLO 10 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] EXECUÇÃO: procurando a origem do alerta... (3/3)

[Agente A] SAÍDA: investigação concluída; retornando à patrulha.

[Agente A] ENTRADA: iniciando patrulha.

===== CICLO 11 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] EXECUÇÃO: patrulhando... (1/3)

===== CICLO 12 =====

[Agente B] EXECUÇÃO: sistema aguardando novo ciclo.

[Agente A] EXECUÇÃO: patrulhando... (2/3)

8. Limitações (se houver – o que você gostaria de ter implementado mas não deu certo).

Após concluir o primeiro ciclo, o alarme fica preso no estado `AlarmeFinalizado` e não volta para `AlarmeInativo`. Faltou criar um evento de reset.

Ajustar a mensagem do `EstadoAlertado` para não dar valores como 3/2 nos logs de execução dos ciclos.

## 9. Referências (se houver – caso use trabalhos de outros autores na execução do trabalho).

A elaboração deste documento e o apoio no desenvolvimento e resolução de bugs do código-fonte utilizaram modelos de IA, mas a estrutura e a arquitetura se baseia no padrão de projeto `State Machines`.